

2^ο Εργαστήριο Αντικειμενοστραφούς Προγραμματισμού

Στόχος της ενότητας αυτής είναι η γνωριμία και εξοικείωση με τις συλλογές αντικειμένων (sequence containers) και πιο συγκεκριμένα με τα μέλη “array”, “vector”, “deque” και “list” της Standard Library της C++.

Sequence containers (array, vector, deque & list)*:

Οι sequence containers είναι συλλογές από ξεχωριστά στοιχεία που μπορεί να προέρχονται είτε από τους απλούς τύπους (int, char κτλ) είτε από πιο σύνθετους όπως αντικείμενα, ενώ κάθε ένα τέτοιο στοιχείο έχει συγκεκριμένη θέση σε σχέση με τα υπόλοιπα. Κοινό χαρακτηριστικό των sequence containers είναι ότι τα στοιχεία τα οποία περιέχουν μπορούν να προσπελαστούν ακολουθιακά (το ένα μετά το άλλο). Ο τρόπος με τον οποίο είναι πραγματικά αποθηκευμένα τα στοιχεία μπορεί να διαφέρει μεταξύ των διαφορετικών sequence containers, καθώς υλοποιούν διαφορετικούς αλγορίθμους αποθήκευσης δεδομένων, γεγονός που μεταφράζεται σε διαφορετικό χρόνο απόκρισης σε αντίστοιχες λειτουργίες (πχ εισαγωγή ενός νέου στοιχείου - insertion) αλλά και σε διαφορετικό απαιτούμενο χώρο μνήμης για την αποθήκευσή τους.

Όπως μπορούμε να δούμε και στον παρακάτω πίνακα, παρότι μεγάλος αριθμός μεθόδων είναι κοινός κάθε sequence container διαφέρει σε σχέση με τα υπόλοιπα. Ο παρακάτω πίνακας έχει συγκεντρωμένες τις περισσότερες από τις μεθόδους των sequence container με τα οποία θα ασχοληθούμε. Μάλιστα, πολλές από αυτές υπάρχουν και στα “string” με τα οποία ασχοληθήκαμε στο προηγούμενο εργαστήριο και η χρήση τους είναι παρόμοια, καθώς και τα string αποτελούν ένα ιδιαίτερο sequence container αποτελούμενο αποκλειστικά από χαρακτήρες.

	array	vector	deque	list	Περιγραφή
	(implicit)	operator=	operator=	operator=	Αναθέτει τιμή στην συλλογή
	N/A	assign	assign	assign	Αναθέτει τιμή στην συλλογή
Element Access	at	at	at	N/A	Προσπελαίνει συγκεκριμένο στοιχείο με έλεγχο ορίων
	operator[]	operator[]	operator[]	N/A	Προσπελαίνει συγκεκριμένο στοιχείο χωρίς έλεγχο ορίων
	front	front	front	front	Προσπελαίνει το πρώτο στοιχείο της συλλογής
	back	back	back	back	Προσπελαίνει το τελευταίο στοιχείο της συλλογής
	data	data	N/A	N/A	Προσπελαίνει το υποκείμενο array
Capacity	empty	empty	empty	empty	Ελέγχει αν η συλλογή είναι άδεια
	size	size	size	size	Επιστρέφει τον αριθμό των στοιχείων της συλλογής
	max_size	max_size	max_size	max_size	Επιστρέφει τον μέγιστο πιθανό αριθμό στοιχείων της συλλογής
	N/A	capacity	N/A	N/A	Επιστρέφει τον αριθμό των στοιχείων που μπορούν να διατηρηθούν στον χώρο της μνήμης που έχει δεσμευτεί μέχρι στιγμής από την συλλογή

Modifiers	N/A	clear	clear	clear	Αδειάζει τη συλλογή
	N/A	insert	insert	insert	Εισάγει στοιχεία
	N/A	erase	erase	erase	Διαγράφει στοιχεία
	N/A	N/A	push_front	push_front	Εισάγει στοιχεία στην αρχή της συλλογής
	N/A	N/A	pop_front	pop_front	Διαγράφει το πρώτο στοιχείο της συλλογής
	N/A	push_back	push_back	push_back	Εισάγει στοιχεία στο τέλος της συλλογής
	N/A	pop_back	pop_back	pop_back	Διαγράφει το τελευταίο στοιχείο της συλλογής
	N/A	resize	resize	resize	Αλλάζει τον αριθμό των στοιχείων που μπορούν να αποθηκευτούν
	N/A	swap	swap	swap	Ανταλλάσσει τα στοιχεία της συλλογής με μία άλλν

Σε κάθε βήμα του συγκεκριμένου εργαστηρίου θα εξετάσουμε και από ένα sequence container, επισημαίνοντας τα ιδιαίτερα χαρακτηριστικά του και τις διαφορές του με τα υπόλοιπα.

* Βασισμένο στο www.cplusplus.com

Βήμα 1^ο - Array

Τα arrays είναι αμετάβλητα προσδιορισμένου μεγέθους sequence container. Εσωτερικά ένα array δεν συγκρατεί κανένα άλλο δεδομένο εκτός από τα στοιχεία που περιέχει (ούτε το μέγεθος, που είναι γνωστό και σταθερό από την κατασκευή του array). Είναι τόσο επαρκές σε όρους χώρου αποθήκευσης όπως ένα κοινό array που δηλώνεται με τις απλές αγκύλες ([]). Το αντικείμενο αυτό προσθέτει μερικώς ένα επίπεδο από γενικές μεθόδους (global functions) καθώς και κάποιες άλλες ιδιότητες, έτσι ώστε τα arrays να μπορούν να χρησιμοποιηθούν σαν στάνταρντ συλλογές (standard containers). Τα arrays θα τα μελετήσουμε για εκπαιδευτικούς λόγους μιας και που τα υπόλοιπα sequence container είναι αυτά που θα χρησιμοποιούμε ως επί το πλείστον στην πράξη.

Στο βήμα αυτό θα διερευνήσουμε τις παρακάτω μεθόδους/συναρτήσεις:

<code>reference array::operator[](size_t pos);</code>	Επιστρέφει μία αναφορά στο στοιχείο που βρίσκεται σε μία συγκεκριμένη θέση (την pos) του array.
<code>reference array::at (size_t pos);</code>	Η λειτουργία της είναι σχεδόν πανομοιότυπη με αυτή της <code>array::operator[]</code> . Η κύρια διαφορά τους είναι στον τρόπο σύνταξης και στο γεγονός ότι η at ελέγχει αν η pos είναι εκτός των ορίων του array, οπότε επιστρέφεται λάθος.
<code>reference array:: front ();</code>	Επιστρέφει μία αναφορά στο πρώτο στοιχείο του array. Αν κληθεί σε ένα άδειο array θα προκληθεί αδιευκρίνιστη συμπεριφορά.
<code>reference array:: back ();</code>	Επιστρέφει μία αναφορά στο τελευταίο στοιχείο του array. Αν κληθεί σε ένα άδειο array θα προκληθεί αδιευκρίνιστη συμπεριφορά.
<code>value_type* array:: data ();</code>	Επιστρέφει έναν δείκτη στο πρώτο στοιχείο του array.
<code>bool array::empty ();</code>	Επιστρέφει bool τιμή ανάλογα με το αν το array είναι άδειο, αν δηλαδή το μήκος είναι 0, διαφορετικά επιστρέφει 1. Η συνάρτηση αυτή δεν τροποποιεί την τιμή του array με κανένα τρόπο.
<code>size_t array::size ();</code>	Επιστρέφει αριθμό των στοιχείων του array. Σε αντίθεση με τον τελεστή sizeof , που επιστρέφει τον αριθμό των εν χρήσει bytes, η μέθοδος αυτή επιστρέφει το μήκος του array σε όρους αριθμού στοιχείων.
<code>size_t array::max_size ();</code>	Επιστρέφει τον μέγιστο αριθμό στοιχείων που μπορεί να φιλοξενήσει το array. Αυτό είναι το μέγιστο δυνητικό μήκος που μπορεί να φτάσει το array λόγω γνωστών περιορισμών από το σύστημα ή τη βιβλιοθήκη, χωρίς να είναι εγγυημένο ότι το array μπορεί να φτάσει αυτό το μήκος.
<code>void array::swap (array& x, array& y);</code>	Ανταλλάσσει το περιεχόμενο δύο arrays x και y .

Θα γνωρίσουμε της μεθόδους της array μέσα από ένα παράδειγμα: Έστω ότι η αεροπορική εταιρεία diAir αποκτά το πρώτο αεροπλάνο της. Πρόκειται για ένα 16/θέσιο αεροπλάνο με 8 σειρές των 2 καθισμάτων. Ο προγραμματιστής της εταιρείας καταχωρεί τα ονόματα των 15 επιβατών του δοκιμαστικού ταξιδιού, και κάνει παράλληλα και διάφορους ελέγχους, ενώ επειδή ο επιβάτης της θέσης 13 αρρώστησε τον αντικατέστησε κάποιος άλλος, όμως αρρώστησε και αυτός. Οπότε για προληπτικούς λόγους η θέση 13 έμεινε κενή. Οι ατυχίες δεν σταμάτησαν εκεί καθώς και το αεροσκάφος αποδείχθηκε ελαττωματικό οπότε ο πωλητής υποχρεωμένος από το συμβόλαιο αγοράς, το αντικατέστησε με άλλο ακριβώς ίδιο.

Πληκτρολογήστε τον παρακάτω κώδικα και ελέγξτε το αποτέλεσμα.

```
1  #include <iostream>
2  #include <cstring>
3  #include <array>
4
5  using namespace std;
6
7  int main ()
8  {
9      array<char*,16> sx_uoa {"Jim","Hellen","Teo","Nik","Cleo","Bill","Simon","John","Marry","Athena","Mike","Paolo","Julia","Son","Terry",""};
10     cout << "First place belongs to: " << sx_uoa.front() << ", whereas the last one belongs to: " << sx_uoa.back() << '\n';
11     cout << "size of sx_uoa: " << sx_uoa.size() << '\n';
12     cout << "max_size of sx_uoa: " << sx_uoa.max_size() << '\n';
13
14     const char* cstr = "At 13th seat was: ";
15     array<char,18> chararray;
16     memcpy (chararray.data(),cstr,18);
17     cout << chararray.data() << sx_uoa[12] << '\n';
18     sx_uoa.at(12)="";
19
20     array<char*,0> sx_dit {}; // Sto prwto bnma 8este array<char*,0> sx_dit {}; k sto deutero array<char*,16> sx_dit {};
21     cout << "SX-UOA " << (sx_uoa.empty() ? "is empty" : "is not empty") << '\n';
22     cout << "SX-DIT " << (sx_dit.empty() ? "is empty" : "is not empty") << '\n';
23     return 0;
24 }
```

Στην συνέχεια αντικαταστήστε το νούμερο "0" της γραμμής 20 με "16", προσθέστε τον παρακάτω κώδικα πριν την εντολή "return 0;" και ελέγξτε το αποτέλεσμα:

```
23  if (sx_dit.empty()){
24      cout << "Dev mporei to aeroskafos va mnv exei 8eseis ka8olou!";
25  }else{
26      sx_uoa.swap(sx_dit);
27      cout << "Meta tnv allagn aeroskafous oi epibates eivai oi parakatw:" << '\n';
28      for (int i=0; i<16;i++){
29          cout << "Stnv 8esn " << i+1 << "ka8etai o/n " << sx_dit[i] << '\n';
30      }
31  }
```

Βήμα 2^ο – Vector

Τα vectors είναι ακολουθίες στοιχείων που μπορούν να αλλάξουν μέγεθος, πράγμα που σημαίνει ότι τα στοιχεία τους μπορούν να προσπελούνται με την ίδια ευκολία που προσπελούνται και στα arrays. Όμως αντίθετα με τα arrays, το μέγεθός τους μπορεί να μεταβληθεί δυναμικά, με την διαχείριση της απαραίτητης μνήμης να ρυθμίζεται αυτόματα από την συλλογή (container).

Εσωτερικά ένα vector χρησιμοποιεί ένα δυναμικά καταναμημένο array για την αποθήκευση των στοιχείων του. Το array αυτό πιθανόν να χρειαστεί κάποια στιγμή να ανακατατατανηθεί ώστε να αυξηθεί σε μέγεθος, καθώς νέα στοιχεία εισέρχονται, που πιθανόν να σημαίνει την δημιουργία ενός νέου array και την μεταφορά όλων των στοιχείων από το παλιό στο νέο. Η διαδικασία αυτή είναι σχετικά δαπανηρή σε όρους χρόνου επεξεργασίας, και για τον λόγο αυτό δεν γίνεται ανακατανομή κάθε φορά που προστίθεται ένα νέο στοιχείο. Αντίθετα με τα arrays, τα vectors πιθανόν να καταλαμβάνουν περισσότερο χώρο για να μπορούν εύκολα να επεκταθούν, οπότε είναι πιθανόν ένα vector με τον ίδιο αριθμό στοιχείων με ένα array να έχει **capacity** μεγαλύτερο από τον χώρο που αυστηρά απαιτείται για την αποθήκευση των στοιχείων (π.χ. το **size** του).

Οι βιβλιοθήκες μπορούν να εφαρμόσουν διάφορες στρατηγικές για την επέκταση του vector και την εξισορρόπηση μεταξύ της χρήσης μνήμης και τον αριθμό ανακατανομών, αλλά σε κάθε περίπτωση, ανακατανομή πρέπει να συμβεί μόνο σε λογαριθμικά αυξανόμενο μέγεθος, έτσι ώστε η εισαγωγή των επιμέρους στοιχείων στο τέλος του φορέα να μπορεί να παρέχεται σε amortized constant time complexity (βλέπε `push_back`). Ως εκ τούτου, σε σύγκριση με τα arrays, τα vectors καταναλώνουν περισσότερη μνήμη, σε αντάλλαγμα την ικανότητα τους να διαχειριστούν την αποθήκευση και δυναμική τους αύξηση με αποτελεσματικό τρόπο.

Σε σύγκριση με τα άλλα δυναμικά sequence containers (deque, lists που θα δούμε στη συνέχεια), τα vectors προσφέρουν πολύ αποδοτική πρόσβαση στα στοιχεία τους (ακριβώς όπως τα arrays) και σχετικά αποτελεσματική επέκταση ή αφαίρεση στοιχείων από το άκρο του. Για τις εργασίες που περιλαμβάνουν την εισαγωγή ή την αφαίρεση στοιχείων σε άλλες θέσεις εκτός από το τέλος, έχουν χειρότερες επιδόσεις από τα άλλα sequence containers.

Όπως φαίνεται και στον παρακάτω πίνακα, μεγάλο μέρος των μεθόδων είναι πανομοιότυπο με αυτό των arrays, γι' αυτό και στο παράδειγμα θα ασχοληθούμε κυρίως με τις μεθόδους που διαφέρουν ή δεν υπάρχουν στα arrays.

Στο βήμα αυτό θα διερευνήσουμε τις παρακάτω μεθόδους/συναρτήσεις:

	<code>vector& vector::operator=(const vector& x);</code>	Εκχωρεί νέα περιεχόμενα στο vector, αντικαθιστώντας τα τρέχοντα.
Element Access	<code>reference vector::operator[](size_t pos);</code>	Ίδιο με array.
	<code>reference vector::at (size_t pos);</code>	Ίδιο με array.
	<code>reference vector:: front ();</code>	Ίδιο με array.
	<code>reference vector:: back ();</code>	Ίδιο με array.
	<code>value_type* vector:: data ();</code>	Ίδιο με array.

Capacity	<code>bool vector::empty ();</code>	Ίδιο με array.
	<code>size_type vector::size ();</code>	Ίδιο με array.
	<code>size_t vector::max_size ();</code>	Ίδιο με array.
	<code>void vector::resize (size_type n, value_type val = value_type());</code>	Επαναπροσδιορίζει το μέγεθος του vector ώστε να χωράει <i>n</i> στοιχεία.
	<code>size_type vector::capacity ();</code>	Επιστρέφει το μέγεθος του χώρου που καταλαμβάνει το vector σε αριθμό στοιχείων. Το <i>vector::capacity</i> δεν είναι απαραίτητα ίσο με το μέγεθος της <i>vector::size</i> . Μπορεί να είναι ίσο ή μεγαλύτερο, με τον extra χώρο να επιτρέπει στο αντικείμενο εύκολη επέκταση. Η διαδικασία επέκτασης του <i>capacity</i> γίνεται και αυτόματα όταν είναι απαραίτητο, ενώ το θεωρητικό μέγιστο ορίζεται από την τιμή του <i>max_size</i> .
	<code>void vector::reserve (size_type n);</code>	Αιτείται το μέγεθος του vector <i>capacity</i> να είναι αρκετό ώστε να χωράνε <i>n</i> στοιχεία (ή περισσότερα).
Modifiers	<code>void vector::assign (size_type n, const value_type& val);</code>	Εκχωρεί νέα περιεχόμενα στο vector, αντικαθιστώντας τα τρέχοντα και μεταβάλλοντας το μέγεθός του ανάλογα.
	<code>void vector::push_back (const value_type& val);</code>	Προσθέτει ένα νέο στοιχείο στο τέλος του vector και αντιγράφει το περιεχόμενο του <i>val</i> στο νέο στοιχείο.
	<code>void vector::pop_back ();</code>	Αφαιρεί το τελευταίο στοιχείο του vector, μειώνοντας και το <i>size</i> κατά ένα.
	<code>void vector::insert (iterator pos, size_type n, const value_type& val);</code>	Το vector επεκτείνεται με την εισαγωγή νέων στοιχείων πριν το στοιχείο της θέσης <i>pos</i> , ενώ το <i>size</i> αυξάνεται σύμφωνα με τον αριθμό των στοιχείων που προστίθενται.
	<code>iterator vector::erase (iterator position);</code> <code>iterator vector::erase (iterator first, iterator last);</code>	Αφαιρεί από το vector ένα μόνο στοιχείο ή ένα εύρος στοιχείων, μειώνοντας και το <i>size</i> ανάλογα.
	<code>void vector::swap (vector& x, vector& y);</code>	Ίδιο με array.
	<code>void vector::clear ();</code>	Αφαιρεί από το vector όλα τα στοιχεία, μειώνοντας και το <i>size</i> στο 0.

Θα γνωρίσουμε της μεθόδους της vector μέσα από την εξέλιξη του προηγούμενου παραδείγματος:

Μετά από ένα διάστημα επιτυχούς χρήσης του αεροπλάνου SX-DIT ήρθε η ώρα η αεροπορική εταιρεία diAir να αποκτήσει το δεύτερο αεροπλάνο της. Η μέχρι τώρα εμπειρία της έδειξε ότι το μεγαλύτερο πρόβλημα ήταν οι διαμαρτυρίες των επιβατών ότι δεν μπορούσαν να μεταφέρουν αποσκευές. Έτσι η diAir αποφάσισε το νέο αεροπλάνο να είναι μεγαλύτερο (30/θέσιο με 10 σειρές καθισμάτων των 3 καθισμάτων) και να μην έχει fix θέσεις. Δηλαδή από τις 10 σειρές καθισμάτων το πολύ οι 3 τελευταίες να μπορούν εύκολα να μετατραπούν σε

θέσεις-χώρους αποσκευών (για την ευκολία τους παραδείγματος θα τις θεωρήσουμε θέσεις αποσκευών), ανάλογα με τις απαιτήσεις, αλλά μπορούν και να αφαιρεθούν τελείως. Δηλαδή η καμπίνα του αεροσκάφους να μπορεί να πάρει πολλές διαμορφώσεις (7 σειρές καθισμάτων και 3 αποσκευών, 8 καθισμάτων και 2 αποσκευών ... 7 μόνο καθισμάτων). Ένα άλλο πρόβλημα που έχει πλέον να αντιμετωπίσει ο προγραμματιστής είναι ότι οι πελάτες/επιβάτες με τον καιρό γίνανε πιο απαιτητικοί, με αποτέλεσμα να πρέπει να κάνει συχνές αλλαγές στον τρόπο που κάθονται οι επιβάτες (διαμόρφωση) καθώς και ότι συναγωνίζονται για θέση με παράθυρο, ενώ κάποιοι πολύ απαιτητικοί ζητάνε να πληρώσουν περισσότερα για να γίνει το δικό τους.

Πληκτρολογήστε τον παρακάτω κώδικα και ελέγξτε το αποτέλεσμα.

```
1  #include <iostream>
2  #include <string>
3  #include <vector>
4
5  using namespace std;
6
7  int main ()
8  {
9      int arxikos_ari8mos_epibatwv = 15;
10     int avamevomevos_ar_epibatwv = 19;
11     vector <string> diamorfwsn1;
12     diamorfwsn1.push_back("Jim");
13     diamorfwsn1.push_back("Hellen");
14     diamorfwsn1.push_back("Teo");
15     diamorfwsn1.push_back("Nik");
16     diamorfwsn1.push_back("Cleo");
17     diamorfwsn1.push_back("Simon");
18     diamorfwsn1.push_back("John");
19     diamorfwsn1.push_back("Marry");
20     diamorfwsn1.push_back("Athena");
21     diamorfwsn1.push_back("Mike");
22     diamorfwsn1.push_back("Paolo");
23     diamorfwsn1.push_back("Julia");
24     diamorfwsn1.push_back("Son");
25     diamorfwsn1.push_back("Terry");
26     diamorfwsn1.push_back("Bob");
27     diamorfwsn1.push_back("Mitch");
28     diamorfwsn1.push_back("Peter");
29     diamorfwsn1.push_back("Deby");
30     vector <string> teliknDiamorfwsn;
31     cout << "Stnv diamorfwsn 1 to size eivai: " << diamorfwsn1.size() << ", to capacity : " << diamorfwsn1.capacity() << " & to max_size: " << diamorfwsn1.max_size() << '\n';
32     if (avamevomevos_ar_epibatwv!=arxikos_ari8mos_epibatwv){
33         diamorfwsn1.resize(avamevomevos_ar_epibatwv);
34         cout << "Pleov stnv diamorfwsn 1 to size eivai: " << diamorfwsn1.size() << ", to capacity: " << diamorfwsn1.capacity() << " & to max_size: " << diamorfwsn1.max_size() << '\n';
35     }
36
37     teliknDiamorfwsn=diamorfwsn1;
38     diamorfwsn1.clear();
39     cout << "diamorfwsn1 " << (diamorfwsn1.empty() ? "is empty" : "is not empty") << '\n';
40     cout << "teliknDiamorfwsn " << (teliknDiamorfwsn.empty() ? "is empty" : "is not empty") << '\n';
41
42     return 0;
43 }
```


Ένας αργοπορημένος επιβάτης ο Bill θέλει πάση θυσία να καθίσει στην θέση 6 που είναι και η αγαπημένη του καθώς διαθέτει παράθυρο και πληρώνει όσο όσο. Αντίθετα ο προηγούμενος επιβάτης της θέσης 6 διαμαρτυρόμενος απαιτεί επιστροφή χρημάτων και αρνείται να πετάξει. Τέλος, η επιβατίδα της τελευταίας θέσης εντοπίστηκε μεθυσμένη και ο κυβερνήτης της απέβαλε. Εισάγετε τον παρακάτω κώδικα στην γραμμή 36, και ελέγξτε το αποτέλεσμα.

```
36 //Neos epibatns o "Bill" pou pasn 8usia 8elei va katsei stnv 8esn 6 k o epibatns tns 6 va paei sto telos h av dev xwraei va mnv petazei.
37 cout << "O palios epibatns tns 8esns 6 eivai o/n: " << diamorfwsn1.at(5) << '\n';
38 deque<string>::iterator iter=diamorfwsn1.begin();
39 for (; iter != diamorfwsn1.end(); ++iter){
40     if (*iter==diamorfwsn1.at(5) break;
41 }
42 diamorfwsn1.insert(iter,"Bill");
43 cout << "O veos epibatns tns 8esns 6 eivai o/n: " << diamorfwsn1.at(5) << '\n';
44 //O epibatns Mike arrwstnse k xavei tnv ptnsn
45 for (iter=diamorfwsn1.begin(); iter != diamorfwsn1.end(); ++iter){
46     if (*iter=="Mike") break;
47 }
48 diamorfwsn1.erase(iter);
49 //H epibatns Deby pou eir8e teleutaia ntav me8usmevn k o pilotos epbale tnv apomakruvsn tns!
50 diamorfwsn1.pop_back();
```

Βήμα 3^ο – Deque (Double Ended Queue)

Τα dequeues είναι ακολουθίες στοιχείων που μπορούν να αλλάξουν μέγεθος, πράγμα που σημαίνει ότι τα στοιχεία τους μπορούν να προσπελαύνονται με την ίδια ευκολία που προσπελαύνονται και στα arrays. Όμως αντίθετα με τα arrays, το μέγεθός τους μπορεί να μεταβληθεί δυναμικά και προς τις δύο κατευθύνσεις, με την διαχείριση της απαραίτητης μνήμης να ρυθμίζεται αυτόματα από την συλλογή (container).

Οι διάφορες βιβλιοθήκες μπορεί να έχουν υλοποιήσει την deque με διάφορους τρόπους, γενικά όμως σαν μια μορφή δυναμικού array. Έτσι παρέχουν μία λειτουργικότητα παρόμοια με αυτή των vectors, αλλά με αποδοτικό insertion και deletion στοιχείων τόσο στην αρχή της συλλογής όσο και στο τέλος. Όμως αντίθετα με τα vectors, για τα dequeues δεν υπάρχει εγγύηση ότι όλα τα στοιχεία θα αποθηκευτούν σε συνεχείς θέσεις στη μνήμη.

Τόσο τα vectors, όσο και τα dequeues παρέχουν μία παρόμοια διεπαφή και μπορούν να χρησιμοποιηθούν για παρόμοιους σκοπούς. Όμως εσωτερικά λειτουργούν με διαφορετικούς τρόπους: ενώ τα vectors χρησιμοποιούν ένα απλό array που χρειάζεται τακτικά επανατοποθέτηση για πιθανές επεκτάσεις, τα στοιχεία μίας deque μπορεί να είναι σκορπισμένα σε διαφορετικά σημεία της μνήμης, με τη συλλογή να κρατάει την απαραίτητη πληροφορία εσωτερικά έτσι ώστε να παρέχει απευθείας πρόσβαση σε οποιοδήποτε στοιχείο σε σταθερό χρόνο και με ομογενοποιημένη διεπαφή συλλογής. Έτσι τα dequeues είναι λίγο πιο πολύπλοκα εσωτερικά από τα vectors, γεγονός που τους επιτρέπει να επεκτείνονται πιο αποδοτικά κάτω από συγκεκριμένες συνθήκες.

Τέλος για λειτουργίες που εμπεριέχουν συχνές εισαγωγές ή διαγραφές στοιχείων, εκτός από το τέλος και την αρχή, τα dequeues αποδίδουν χειρότερα από τις lists.

Όπως φαίνεται και στον παρακάτω πίνακα, μεγάλο μέρος των μεθόδων είναι πανομοιότυπο με αυτό των vectors, γι' αυτό και στο παράδειγμα θα ασχοληθούμε κυρίως με τις μεθόδους που διαφέρουν ή δεν υπάρχουν στα vectors.

Στο βήμα αυτό θα διερευνήσουμε τις παρακάτω μεθόδους/συναρτήσεις:

Element Access	<code>deque& deque::operator=(const deque& x);</code>	Ίδιο με vector.
	<code>reference deque::operator[](size_t pos);</code>	Ίδιο με vector.
	<code>reference deque::at (size_t pos);</code>	Ίδιο με vector.
	<code>reference deque:: front ();</code>	Ίδιο με vector.
	<code>reference deque:: back ();</code>	Ίδιο με vector.
Capacity	<code>bool deque::empty ();</code>	Ίδιο με vector.
	<code>size_type deque::size ();</code>	Ίδιο με vector.
	<code>size_t deque::max_size ();</code>	Ίδιο με vector.
	<code>void deque::resize (size_type n, value_type val = value_type());</code>	Ίδιο με vector.

Modifiers	<code>void deque::assign (size_type n, const value_type& val);</code>	Ίδιο με vector.
	<code>void deque::push_back (const value_type& val);</code>	Ίδιο με vector.
	<code>void deque::push_front (const value_type& val);</code>	Προσθέτει ένα νέο στοιχείο στην αρχή του deque και αντιγράφει το περιεχόμενο του val στο νέο στοιχείο.
	<code>void deque::pop_back ();</code>	Ίδιο με vector.
	<code>void deque::pop_front ();</code>	Αφαιρεί το πρώτο στοιχείο του deque, μειώνοντας και το size κατά ένα.
	<code>void deque::insert (iterator pos, size_type n, const value_type& val);</code>	Ίδιο με vector.
	<code>iterator deque::erase (iterator position);</code> <code>iterator deque::erase (iterator first, iterator last);</code>	Ίδιο με vector.
	<code>void deque::swap (deque& x, deque& y);</code>	Ίδιο με vector.
	<code>void deque::clear ();</code>	Ίδιο με vector.

Θα γνωρίσουμε της μεθόδους της deque μέσα από την εξέλιξη του προηγούμενου παραδείγματος:

Με την καθιέρωση της εταιρείας diAir και την τακτική χρήση των δρομολογίων της από κάποιους πελάτες, αυτοί έθεσαν το θέμα της δημιουργίας κάποιων προνομίων και κυρίων την δημιουργία vip θέσεων. Οι εταιρεία λοιπόν αποφάσισε να ικανοποιήσει το αίτημα των τακτικών πελατών της και έδωσε πλέον την δυνατότητα οι πρώτες σειρές καθισμάτων να μπορούν να χρησιμοποιηθούν ως θέσεις vip. Χρησιμοποιώντας τον κώδικα του βήματος 2, αντιγράψτε τον σε ένα νέο αρχείο, αντικαταστήστε την λέξη **vector** με την **deque** και εισάγετε πριν την εντολή “`teliknDiamorfwsn=diamorfwsn1;`” τον παρακάτω κώδικα:

Τροποποιήστε κατάλληλα τον παραπάνω κώδικα (copy/paste/amend) ώστε να πάρει την παρακάτω μορφή:

```

52 //Neoi vip epibates "Suzan" kai "Gearge".
53 diamorfwsn1.push_front("Gearge");
54 diamorfwsn1.push_front("Suzan");
55 //Epeidn n Suzan eixe sumptwmata me8ns o Kubervntns apofasise tnv apomakruvsn tns
56 diamorfwsn1.pop_front();

```

Βήμα 4^ο – list:

Τα lists είναι sequence containers που επιτρέπουν εισαγωγή και διαγραφή στοιχείων οπουδήποτε εντός της συλλογής σε σταθερό χρόνο, αλλά και την διάσχιση της συλλογής και προς τις δύο κατευθύνσεις. Υλοποιούνται ως διπλά διασυνδεδεμένες λίστες στοιχείων, οι οποίες μπορούν να αποθηκεύουν τα στοιχεία που περιέχουν σε διαφορετικές και άσχετες μεταξύ τους θέσεις μνήμης. Η ταξινόμηση διατηρείται εσωτερικά με την διασύνδεση κάθε στοιχείου με το προηγούμενο και το επόμενο.

Σε σύγκριση με τις άλλες συλλογές που εξετάσαμε μέχρι στιγμής (array, vector και deque), τα lists αποδίδουν καλύτερα στην εισαγωγή, εξαγωγή και μετακίνηση στοιχείων σε οποιαδήποτε θέση μέσα στην συλλογή για την οποία υπάρχει επαναλήπτης (iterator, που αφορά την συγκεκριμένη θέση). Για τον λόγω αυτόν αποδίδουν εξαιρετικά σε αλγορίθμους που κάνουν εκτεταμένη χρήση τέτοιων μετακινήσεων, όπως οι αλγόριθμοι ταξινόμησης. Το μόνο μειονέκτημα των lists είναι ότι δεν υπάρχει δυνατότητα απευθείας πρόσβασης στα στοιχεία τους, όταν γνωρίζουμε την θέση τους. Για παράδειγμα, η πρόσβαση στο έκτο στοιχείο ενός list, μπορεί να γίνει με την χρήση ενός iterator από μία γνωστή θέση (όπως η αρχή και το τέλος της συλλογής) προς την θέση του στοιχείου που μας ενδιαφέρει, διαδικασία που απαιτεί γραμμικό χρόνο. Επίσης απαιτείται extra μνήμη για την διατήρηση της διαδικασίας διασύνδεσης των στοιχείων μεταξύ τους, που για μεγάλο αριθμό στοιχείων μπορεί να αποτελεί ένα σημαντικό ποσοστό για μεγάλες λίστες με μικρού μεγέθους στοιχεία.

Όπως φαίνεται και στον παρακάτω πίνακα, μεγάλο μέρος των μεθόδων είναι πανομοιότυπο με αυτό των vectors, γι' αυτό και στο παράδειγμα θα ασχοληθούμε κυρίως με τις μεθόδους που διαφέρουν ή δεν υπάρχουν στα vectors.

Στο βήμα αυτό θα διερευνήσουμε τις παρακάτω μεθόδους/συναρτήσεις:

	<code>list& list::operator=(const list& x);</code>	Ίδιο με deque.
	<code>reference list:: front ();</code>	Ίδιο με deque.
	<code>reference list:: back ();</code>	Ίδιο με deque.
Capacity	<code>bool list::empty ();</code>	Ίδιο με deque.
	<code>size_type list::size ();</code>	Ίδιο με deque.
	<code>size_t list::max_size ();</code>	Ίδιο με deque.
	<code>void list::resize (size_type n, value_type val = value_type());</code>	Ίδιο με deque.
Modifiers	<code>void list::assign (size_type n, const value_type& val);</code>	Ίδιο με deque.
	<code>void list::push_back (const value_type& val);</code>	Ίδιο με deque.
	<code>void list::push_front (const value_type& val);</code>	Ίδιο με deque.
	<code>void list::pop_back ();</code>	Ίδιο με deque.
	<code>void list::pop_front ();</code>	Ίδιο με deque.

Λειτουργίες	<code>void list::insert (iterator pos, size_type n, const value_type& val);</code>	Ίδιο με deque.
	<code>iterator list::erase (iterator position);</code> <code>iterator list::erase (iterator first, iterator last);</code>	Ίδιο με deque.
	<code>void list::swap (list& x, list& y);</code>	Ίδιο με deque.
	<code>void list::clear ();</code>	Ίδιο με deque.
	<code>void list::splice (iterator position, list& x);</code> <code>void list::splice (iterator position, list& x, iterator i);</code> <code>void list::splice (iterator position, list& x, iterator first, iterator last);</code>	Μεταφέρει στοιχεία από την list x , εισάγωντάς τα στην θέση pos . Στην 3 ^η περίπτωση μεταφέρονται στοιχεία εύρους [first , last] από την x . Ουσιαστικά τα στοιχεία αφαιρούνται από την μητρική τους list και εισάγονται στην νέα, με ταυτόχρονη αλλαγή των μεγεθών των δύο συλλογών.
	<code>void list::remove (const value_type& val);</code>	Απομακρύνει από την συλλογή όλα τα στοιχεία με τιμή val , καταστρέφοντας τα στοιχεία αυτά και αλλάζοντας αντίστοιχα το μέγεθος της συλλογής. Αντίθετα με την list::erase στην οποία η θέση του στοιχείου γίνεται με χρήση iterator, η μέθοδος αυτή αφαιρεί τα στοιχεία με βάση την τιμή τους.
	<code>void list::remove_if (Predicate pred);</code>	Λειτουργεί παρόμοια με την list::remove , απομακρύνοντας από την συλλογή όλα τα στοιχεία για τα οποία η Predicate pred επιστρέφει True.
	<code>void unique();</code>	Απομακρύνει όλα τα στοιχεία, εκτός του πρώτου, από μία ομάδα συνεχόμενων ίδιων στοιχείων, γεγονός που καθιστά την συγκεκριμένη μέθοδο ιδιαίτερα χρήσιμη σε ταξινομημένες λίστες.
	<code>void merge (list& x);</code>	Αναμειγνύει την x μέσα στην συλλογή μεταφέροντας όλα τα στοιχεία της στις κατάλληλες θέσεις ώστε να προκύψει ξανά μία ταξινομημένη λίστα, γεγονός που προϋποθέτει ότι και οι δύο αναμειχθείσες λίστες πρέπει να είναι ταξινομημένες. Για διαφορετικό αποτέλεσμα ή μη ταξινομημένες λίστες δείτε την list::splice .
	<code>void sort();</code>	Ταξινομεί να στοιχεία της συλλογής με αυστηρή διάταξη.

Λόγω της μεγάλης ομοιότητας πολλών μεθόδων της list με της συλλογές vector και deque, θα επικεντρωθούμε σε αυτές που δεν υπάρχουν στις προηγούμενες συλλογές. Πληκτρολογήστε τον παρακάτω κώδικα και ελέγξτε το αποτέλεσμα.

```

1  #include <iostream>
2  #include <list>
3
4  using namespace std;
5
6  void printList(std::list<int> alist, int j, string meta_tnv_evtoln){
7      //Tupwvei ta periexomeva tns listas 'alist' me ari8mnsn 'j'
8      if (!alist.empty()){
9          std::list<int>::iterator it;
10         it = alist.begin();
11         ++it;
12         cout << "Meta tnv evtoln " << meta_tnv_evtoln
13             << ",\n\t\t n intList" << j << " periexei:";
14         for (it=alist.begin(); it!=alist.end(); ++it)
15             cout << ' ' << *it;
16         cout << '\n';
17     }else{
18         cout << "H intList" << j << " eivai adeia!\n";
19     }
20 }
21
22 int main ()
23 {
24     list<int> intList1, intList2, intList3, intList4;
25     list<int>::iterator it;
26
27     for (int i=1; i<=4; ++i){
28         intList1.push_back(i);    // intList1: 1 2 3 4
29         intList3.push_back(i);    // intList3: 1 2 3 4
30     }
31
32     for (int i=1; i<=3; ++i){
33         intList2.push_back(i*10); // intList2: 10 20 30
34         intList4.push_back(i*10); // intList4: 10 20 30
35     }
36     printList(intList1,1,"for ... intList1.push_back(i)");
37     printList(intList2,2,"for ... intList2.push_back(i*10)");
38     it = intList1.begin();
39     ++it;    // points to 2

```

```

40
41     intList1.splice (it, intList2); // intList1: 1 10 20 30 2 3 4
42                                     // intList2 (empty)
43                                     // "it" deixvei sto 2 (sto 5to stoixeio)
44
45     intList2.splice (intList2.begin(),intList1, it);
46                                     // intList1: 1 10 20 30 3 4
47                                     // intList2: 2
48                                     // "it" eivai pleov invalid.
49     printList(intList1,1, "intList1.splice (it, intList2)");
50     printList(intList2,2, "intList2.splice (intList2.begin(),intList1, it)");
51     it = intList1.begin();
52     advance(it,3);    // "it" deixvei sto 30
53
54     intList1.splice ( intList1.begin(), intList1, it, intList1.end());
55                                     // intList1: 30 3 4 1 10 20
56     printList(intList1,1,"intList1.splice ( intList1.begin(), intList1, it, intList1.end())");
57     printList(intList2,2,"Kamia");
58
59     intList1.remove(1);
60
61     printList(intList1,1,"intList1.remove(1)");
62
63     intList3.merge(intList4);
64     printList(intList4,4,"intList3.merge(intList4)");
65
66     intList1.merge(intList3);
67     printList(intList1,1,"intList1.merge(intList3)");
68     printList(intList3,3,"intList1.merge(intList3)");
69
70     intList1.unique();
71     printList(intList1,1,"intList1.unique()");
72
73     intList1.sort();
74     printList(intList1,1,"intList1.sort()");
75
76     intList1.unique();
77     printList(intList1,1,"intList1.unique()");
78
79     return 0;
80 }

```

Πληκτρολογήστε τον παρακάτω κώδικα και ελέγξτε το αποτέλεσμα.

```
1  #include <iostream>
2  #include <list>
3
4  using namespace std;
5
6  void printList(list<int> alist, string meta_tnv_evtoln){//Tupwvei ta periexomeva tns listas 'alist' me ari8mnsn 'j'
7      if (!alist.empty()){
8          cout << "Meta tnv evtoln " << meta_tnv_evtoln << ",\n\t\t\t n mylist periexei:";
9          for (list<int>::iterator it=alist.begin(); it!=alist.end(); ++it)
10             cout << ' ' << *it;
11             cout << '\n';
12     }else{
13         cout << "H mylist eivai adeia!\n";
14     }
15 }
16
17 bool single_digit (const int& value) { return (value<10); }
18
19 bool is_odd (const int& value) { return (value%2)==1; }
20
21 int main ()
22 {
23     int myints[] = {15,36,7,17,20,39,4,1};
24     list<int> mylist (myints,myints+8); // 15 36 7 17 20 39 4 1
25
26     printList(mylist," arxikopoints");
27
28     mylist.remove_if (single_digit); // 15 36 17 20 39
29
30     printList(mylist,"mylist.remove_if (single_digit)");
31
32     mylist.remove_if (is_odd); // 36 20
33
34     printList(mylist,"mylist.remove_if (is_odd)");
35
36     return 0;
37 }
```


Βήμα 5^ο – Άσκηση:

Έστω η αεροπορική εταιρεία την οποία είδαμε στα βήματα 1, 2 και 3. Η εταιρεία λοιπόν αγοράζει δύο νέα αεροπλάνα των 30 θέσεων (10 σειρές των 3 καθισμάτων) με ευμετάβλητο αριθμό καθισμάτων και αποσκευών, ενώ και τα καθίσματα μπορεί να αφορούν VIP και standard θέσεις. Αυτό σημαίνει ότι μπορούν να υπάρξουν πολλές και διαφορετικές διαμορφώσεις των αεροσκαφών.

Θεωρώντας ως είσοδο (δεδομένα) τον αριθμό των επιβαινόντων VIP, standard θέσεων και “θέσεων” αποσκευών, αποτυπώστε σε κατάλληλες μεταβλητές τις αντίστοιχες τιμές. Για τις λίστες των ονομάτων των αντίστοιχων επιβατών/αποσκευών (VIP, standard θέσεων και “θέσεων” αποσκευών) να γίνει χρήση της πιο συμφέρουσας για κάθε περίπτωση sequence container (vector, deque ή list).

Τα όρια των τιμών των θέσεων είναι τα παρακάτω:

<i>Τύπος θέσης</i>	<i>min</i>	<i>max</i>	<i>βήμα</i>
VIP	0	6 (2x3)	3
Standard	15 (5x3)	21 (7x3)	3
Baggage	3 (1x3)	9 (3x3)	3
Σειρές καθισμάτων		10	3

Σας δίνονται τα παρακάτω τέσσερα σενάρια. Γράψτε των κατάλληλο κώδικα που να υπολογίζει τη σύνθεση των αεροσκαφών και να λαμβάνει υπόψη τους παρακάτω περιορισμούς:

- Οι VIP θέσεις να καταλαμβάνουν όλοι τη σειρά, όπως και οι standard και οι θέσεις που καταλαμβάνονται από αποσκευές
- Προτεραιότητα είναι οι VIP επιβάτες να μπαίνουν όλοι μαζί (αν γίνεται) στο ίδιο αεροπλάνο.
- Οι αποσκευές να μην μπαίνουν ποτέ όλες μαζί σε ένα αεροσκάφος για λόγους απόδοσης και ασφάλειας των αεροσκαφών.
- Η πτήση ενός αεροσκάφους θα ακυρώνεται όταν έχει να μεταφέρει λιγότερους από 10 επιβάτες συνολικά (vip, std και bags).
- Αν αρκεί το ένα αεροσκάφος να μην γίνεται διπλό δρομολόγιο.

Στο τέλος να εκτυπώνεται η τελική σύνθεση των αεροσκαφών. Στην περίπτωση που δεν χωράνε όλοι οι επιβάτες να εκτυπώνετε ποιοι επιβάτες/αποσκευές δεν χωράνε.

<i>Σενάριο 1</i>	<i>Σενάριο 2</i>	<i>Σενάριο 3</i>	<i>Σενάριο 4</i>
VIPs: vip_0 - vip_11 Std: std_0 – vip_25 Bags: bag_0 – bag_5	VIPs: vip_0 - vip_4 Std: std_0 – vip_35 Bags: bag_0 – bag_7	VIPs: vip_0 - vip_4 Std: std_0 – vip_37 Bags: bag_0 – bag_17	VIPs: vip_0 - vip_3 Std: std_0 – vip_20 Bags: bag_0 – bag_4

Σημείωση: τα ονόματα των επιβατών ή αποσκευών θα μπορούσαν να ξεκινάνε από “vip_” για τους VIP επιβάτες, “std_” για τους επιβάτες των standard θέσεων και για λόγους ευκολίας, τις θέσεις που καταλαμβάνονται από αποσκευές “bag_01, bag_02” κτλ, όπως δίνονται.